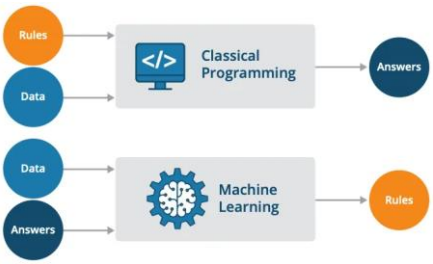
Artificial Intelligence
Introduction and examples

Stefaan Haspeslagh
Vives University of applied sciences – Association KU Leuven – Kortrijk, Belgium

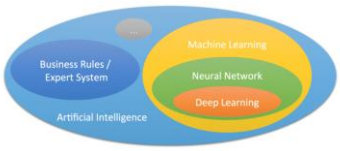
Artificial Intelligence: introduction and examples

STATE SPACE REPRESENTATION

Classical AI vs Machine Learning



- Classical programming and AI (e.g. expert systems):
  - Input: data + **rules**
  - Output: answers
- Machine learning:
  - Input: data + answers (if supervised learning)
  - Output: **rules**
- **Model** = the set of rules used as input (classical AI) or returned as output (ML)
- In this course:
  - Classical search algorithms
  - Model: **state space representation**



In klassieke AI geven we het algoritme data en beslissingsregels mee als input. Het algoritme berekent dan voor ons de nodige resultaten. In machine learning krijgt het algoritme ook data als input. Bij supervised learning geven we ook al de antwoorden mee die we labels noemen. In unsupervised learning krijgt het algoritme enkel data. Het algoritme gaat dan bij zowel supervised als unsupervised learning 'leren' uit de inputdata om zo tot beslissingsregels te komen waarmee we dan voorspellingen kunnen doen op nieuwe data. De verzameling van regels noemen we het model, dat meestal een wiskundig model is.

Het ondubbelzinnig modelleren van problemen is niet iets dat alleen in AI wordt gedaan. Ook in klassiek programmeren doen we dat. Denk maar aan het opstellen van een OO-model of een relationeel datamodel. In het laatste geval gebruiken we dat datamodel om onze data te structureren en in een databank te stoppen, waarna we SQL gebruiken om queries uit te voeren die ons de nodige antwoorden geven.

In deze cursus gaan we dieper in op klassieke zoekalgoritmes om zoekproblemen op te lossen. Deze algoritmen zijn geen machine learning, en dus moeten we eerst een model opstellen om het probleem ondubbelzinnig voor te stellen. Dit doen we a.d.h.v. de state space representation, wat het onderwerp is van dit topic.

State-space representation

- What is state-space representation?
- Which are the technical issues that arise in that context?
- What are the alternatives that the paradigm offers to solve a problem in the state-space representation?
  - What (kind) of algorithms?
  - What about the solution? Is it optimal or 'just good'?
  - Will you find a solution?
  - …

State-space representation is dus een manier om problemen te modelleren zodat een algoritme op zoek kan gaan naar de oplossing van dat probleem.
In de cursus A.I. gaan we voor een aantal schijnbaar eenvoudige problemen zo'n model opstellen en vervolgens oplossen met de juiste algoritmen.
Dit topic gaat voornamelijk over het modelleren, vanaf topic 4 bekijken we verschillende types algoritmen.
In de praktijksessies gaan we een aantal van die algoritmen ook effectief implementeren en testen.

Classical example: the 8-puzzle

1	3	8
2		7
5	4	6

→

1	2	3
4	5	6
7	8	

- Important concepts:
  - State
  - Goal
  - Move

- Question: how to find a sequence of moves to reach the goal from an initial state?

We zullen de “8-puzzle” gebruiken als voorbeeld. De 8-puzzle is een schuifpuzzel waar de bedoeling is om de getallen in de juiste volgorde te plaatsen door vakjes te verschuiven.
Een veel voorkomende variant van deze puzzel is wanneer de getallen vervangen worden door een prent of foto.
➔ Java jar at http://www.aiai.ed.ac.uk/~gwickler/eightpuzzle-uninf.html (fyi)
➔ A* solution in C++ at https://faramira.com/solving-8-puzzle-problem-using-a-star-search-in-c/

Er zijn drie belangrijke concepten die cruciaal zijn om problemen te kunnen modelleren en oplossen.

Concept 1: state
Op de één of andere manier moet je in staat zijn de toestand van een probleem, in dit geval de positie van de vakjes (en bijhorende getallen) in de puzzel, voor de stellen in een programmeertaal. Je kan hiervoor bestaande gegevensstructuren gebruiken (arrays, lijsten, …), of je kan zelf een OO-model maken om het probleem voor te stellen. Afhankelijk van de complexiteit en moeilijkheid vraag het modelleren en voorstellen van toestanden weinig of zeer veel inspanning.

Concept 2: doel
Je moet het doel, of de verschillende doelen, efficiënt kunnen herkennen. Dit wil

zeggen dat je een goede, volledige, beschrijving moet opstellen van die toestanden waarin je de oplossing gevonden hebt, of het spel wint. Voor de 8-puzzle is dit eenvoudig. Er is maar één doeltoestand. Hier volstaat het bijvoorbeeld om voor elk vakje/getal te controleren of het zich op de juiste plaats bevindt.
Bij schaken is dit moeilijker, er zijn meerdere toestanden van pionnen op het spelbord in te beelden waarin je je tegenstander schaakmat zet. Hier moeten we dus andere manieren bedenken om te controleren of een bepaalde bordsituatie een doelsituatie is.

Concept 3: move
Het begrip move wordt uitgelegd aan de hand van een spel zoals de 8-puzzle. Een move is een zet die je kan doen vanuit een bepaalde spelsituatie. Meestal zijn er meerdere moves nodig.
Vanuit de eerste 8-puzzle op deze dia zijn er vier moves mogelijks. Namelijk 3 verschuiven, 2 verschuiven, 7 verschuiven of 4 verschuiven.

State-space representation: general outline

- Select some way to represent states in the problem in an unambiguous way.
- Formulate all actions that can be preformed in states:
  - including their preconditions and effects
    - == PRODUCTION RULES
- Represent the initial state (s).
- Formulate precisely when a state satisfies the goal of our problem.
- Activate the production rules on the initial state and its descendants, until a goal state is reached.

Op deze slide staan de verschillende onderdelen/aspecten die moeten behandeld worden bij een state-space representation.
In de volgende slides komt telkens één of meedere aspecten uitgebreider aan bod.

State-space representation: issues

1	3	8
2		7
5	4	6

- How to represent states? (repr. 1)
  - Using a 3 x 3 matrix
- How to formulate production rules? (repr. 2)
  - Ex.:
    - express how/when squares may be moved?
    - Or: express how/when the blank space is moved?
- When is a rule applicable to a state? (matching)
- How to formulate when the goal criterion is satisfied and how to verify that it is?
- How/which rules to activate? (control)

De eerste vraag die we ons moeten stellen is, hoe stellen we één toestand, één situatie van een probleem voor? Meer bepaald: welke gegevenstructuur gebruiken we?
Voor eenvoudige spelletjes is dit vaak vanzelfspreken, voor de 8-puzzle bijvoorbeeld kunnen we een meerdimensionale 3x3 array gebruiken.

Ten tweede moeten we de regels van het spel uitdrukken. Dus hoe,of wanneer kunnen vierkantjes verplaatst worden. Of anders bekeken, naar waar kan het blanco vakje verplaatst worden?
De regels (production rules) worden dus gebruikt om de moves te vinden die mogelijk zijn vanuit een bepaalde spelsitutatie.

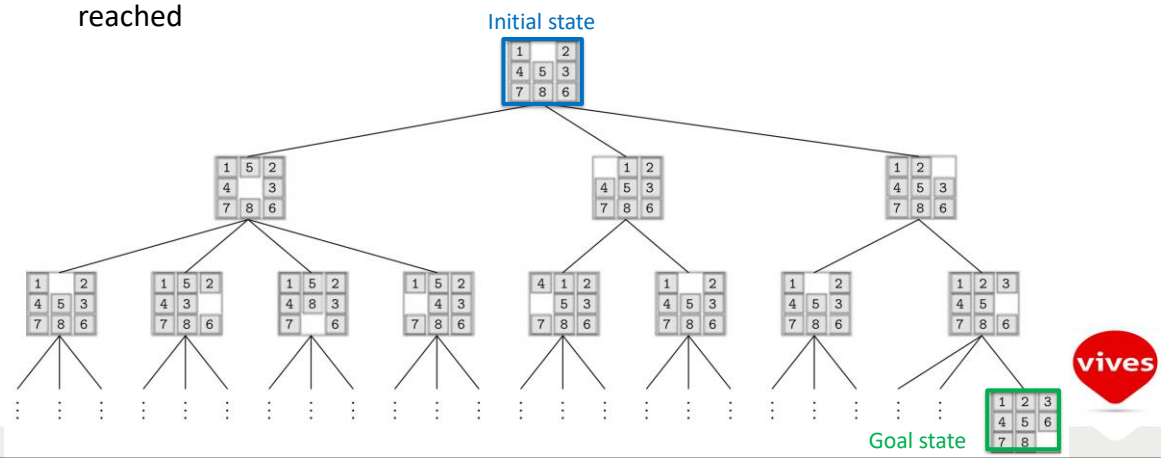
De regels bij 8-puzzle zouden kunnen zijn:
- Vakje naar boven
- Vakje naar beneden
- Vakje naar links
- Vakje naar rechts

Ten derde moeten we voor elke spelbordsituatie kunnen vinden welke regels en dus welke toepasbaar zijn en welke niet. In de figuur op de slide bv, is de move vakje ‘2’ naar beneden niet mogelijk, vakje ‘2’ naar rechts wel.

Als laatste moeten we vanuit elke situatie kunnen beslissen welke regel, en dus welke move, we best uitvoeren om het spel te kunnen winnen. Dit is het control problem.

The search tree

- Each state-space representation defines a search tree
  - From the initial solution, for every (possible) production rule, a new node can be reached



Als we vanuit de beginsituatie alle mogelijke zetten bekijken, en vanuit elke nieuwe spelsituatie opnieuw alle mogelijke zetten berekenen, dan bekomen we een spelboom zoals op de figuur.
Als we op de gegeven spelboom telkens de rechtertak (en dus bijhorende move) kiezen, komen we in de doelsituatie terecht (en winnen we het spel).
De opeenvolgende moves zijn dan:

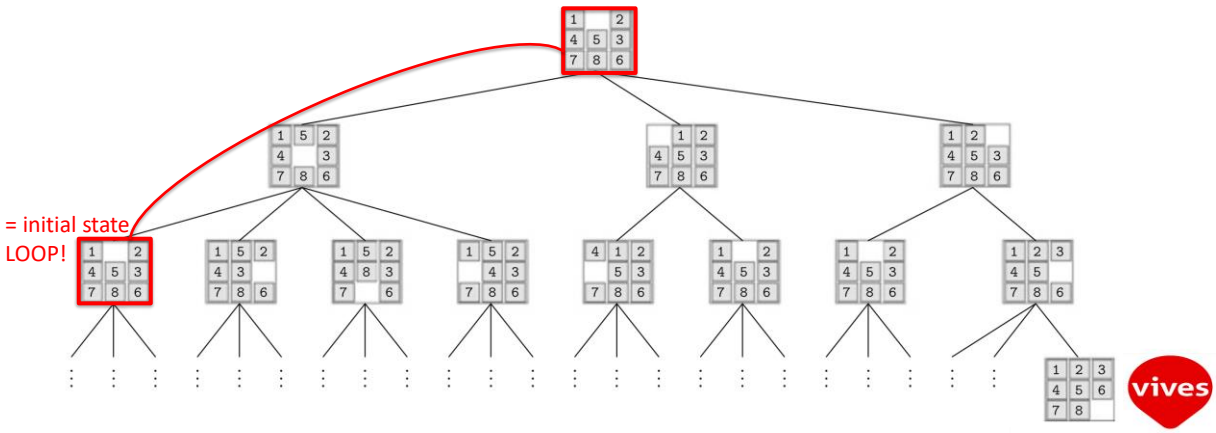
- Vakje 2 naar links
- Vakje 3 naar boven
- Vakje 6 naar boven

Die opeenvolgende moves vormen dus een pad in de spelboom

Zo'n spelboom wordt al snel heel groot (zie volgende slide).
De kunst zal dus zijn om een goed A.I. algoritme te selecteren om de juiste opeenvolging van moves te vinden en dus een pad naar de doelsituatie.

The search tree

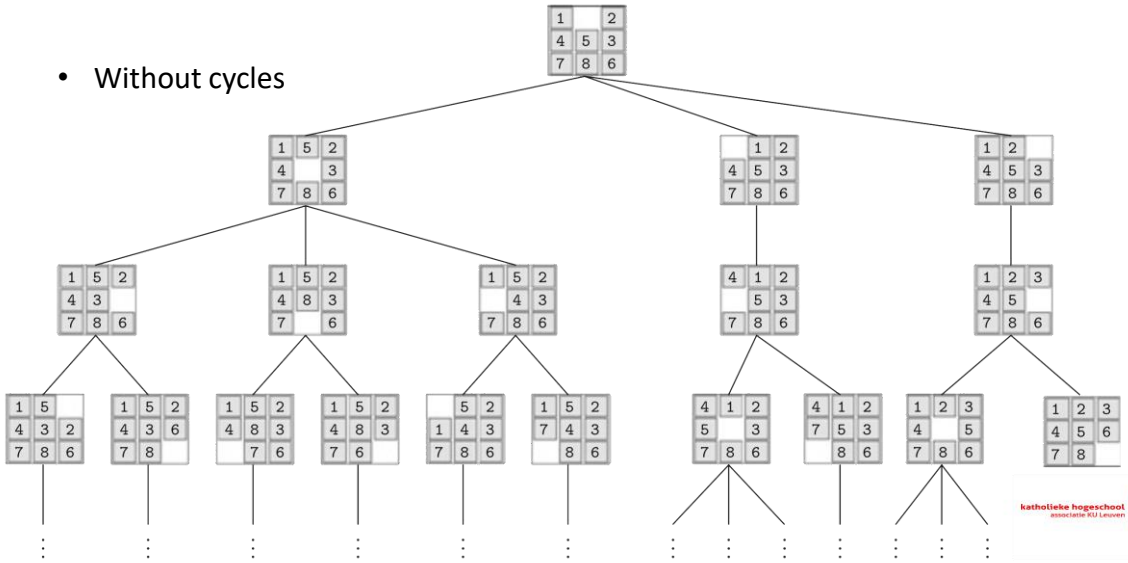
- Cycles!



De grootte van de spelboom bepaalt voor een deel de complexiteit van het spel (en dus de moeilijkheid waarmee een algoritme de oplossing kan vinden).
De grootte van de boom kan beperkt worden door bijvoorbeeld lussen niet toe te laten.
Een lus is een opeenvolging van moves waardoor je terug in een vorige spelsituatie terecht komt, bijvoorbeeld als we telkens de linker tak nemen. Na twee moves komen we terug in de beginsituatie terecht en als we niet oppassen en dit telkens doen, dan komen we in een oneindige lus terecht.
Mensen kunnen heel snel oneindige lussen detecteren in een spelboom zoals die van de 8-puzzle. Voor een algoritme is dat een grotere uitdaging.

The search tree

- Without cycles



De spelboom van de 8-puzzle waar cycles (lussen) weggelaten zijn

The search tree

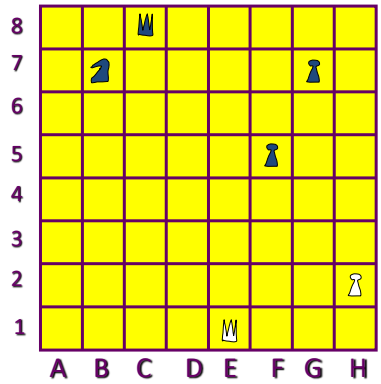
- Is huge!
  - 9!/2 = 181.440 nodes
  - Even for a simple game like the 8-puzzle
- Size depends on branching factor:
  - How many rules, on average, are valid per state?
  - E.g. how many moves can you make from each state?
- Typical values:
  - 8-puzzle: ≈ 1,8
  - chess: 35
  - go: 250

Uitleg staat voldoende in Engels op de slide.

9! Is 9 faculteit, dit is het getal 9 maal 8 maal 7 maal 6 …. Dus 9.8.7.6.5.2.1 = 362880
Je moet zelf niet kunnen uitrekenen of bewijzen waarom de spelboom van de 8-puzzle 9!/2 is.

Chess

- <u>Problem</u>: develop a program that plays chess (well).
  - Step 1: A way to represent board situations in an unambiguous way



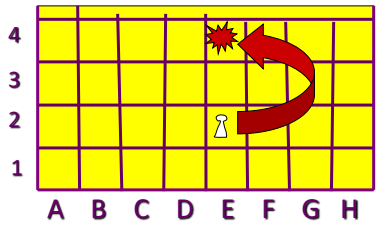
List:
((king_black, 8, C),
(knight_black, 7, B),
(pawn_black, 7, G),
(pawn_black, 5, F),
(pawn_white, 2, H),
(king_white, 1, E))

12

We herhalen de problem stelling voor schaken.
Hier zouden we kunnen opteren voor een 8x8 matrix (tweedimensionale array). We tonen echter even een alternatieve voorstelling.
Je kan bijvoorbeeld voor alle stukken op het bord een lijst bijhouden met de posities.
Voor elk element van de lijst kan je alle stukken overlopen van de speler wiens beurt het is (wit of zwart).
Voor elk stuk kan je dan de nodige production rules toepassen en bijhorende moves genereren (zie volgende slide voor een voorbeeld).

Chess

- Problem: develop a program that plays chess (well).
  - Step 2. Describe the rules that represent allowed moves.
  - ((pawn_white, 2, x), (blank, 3, x), (blank, 4, x)) → add((pawn_white, 4, x)), remove((pawn_white, 2, x))



Op een vorige slide stond dat de productieregels bestaan uit precondities en effecten. Hier zie je een mooi voorbeeld van wat we hiermee bedoelen.

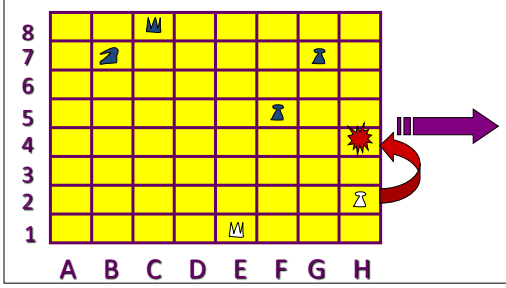
De productieregel is namelijk:

((pawn_white, 2, x), (blank, 3, x), (blank, 4, x)) → add((pawn_white, 4, x)), remove((pawn_white, 2, x))

Het deel voor het pijltje zijn de precondities, het deel achter het pijltje de effecten.

Chess

- Problem: develop a program that plays chess (well).
  - Step 3. Provide a way to check whether a rule is applicable to some state:



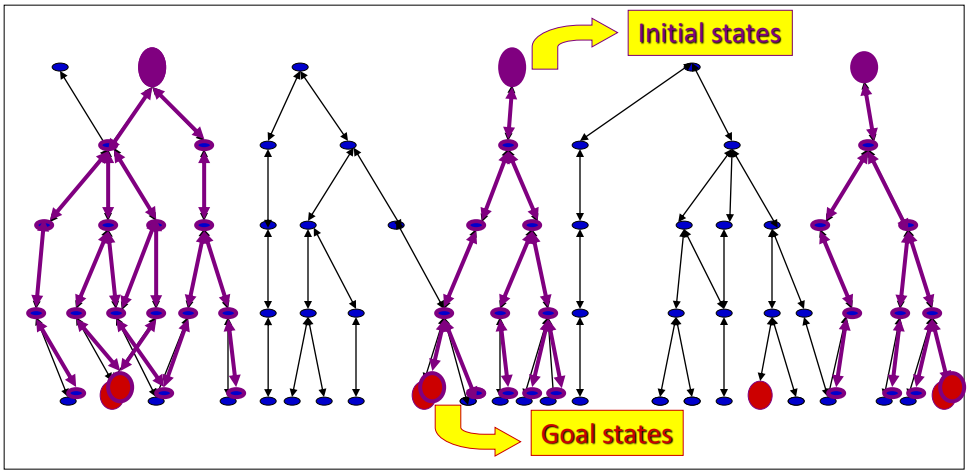
List:
((king_black, 8, C),
(knight_black, 7, B),
(pawn_black, 7, G),
(pawn_black, 5, F),
(pawn_white, 2, H),
(king_white, 1, E))

((pawn_white,2,x) , (blank, 3, x), (blank, 4, x))
→ **add((pawn_white, 4, x)),**
remove((pawn_white, 2, x))

Chess

- Problem: develop a program that plays chess (well).
  - Step 4. How to specify a state in which the goal is reached (= a winning state)
    - win(black) ← attacked(king_white) and no_legal_move(king_white)
    - More than 1 goal state possible!!!
  - Step 5. A mechanism that selects in each state an appropriate rule to apply.
    - E.g. which move to select at each level in the search tree?
    - E.g. what 'path' to select in the search tree?
    - E.g. how to 'search in the search tree'?
    - → ALGORITHMS

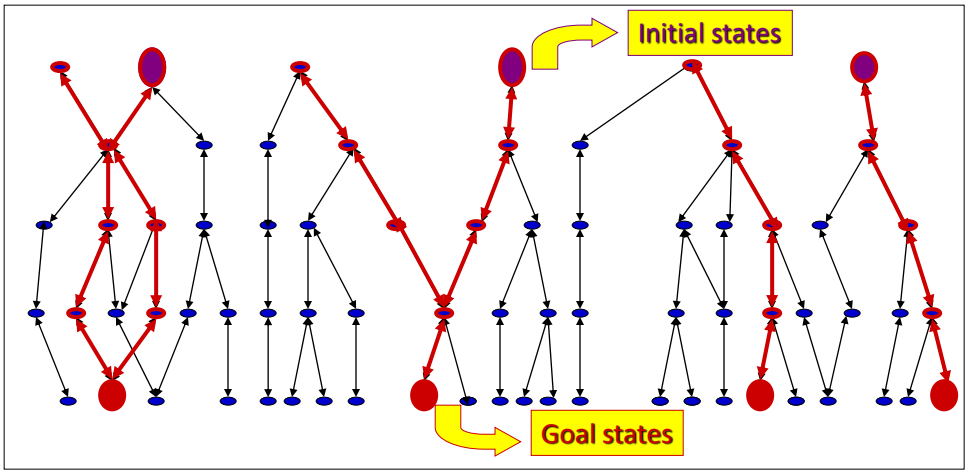
Forward versus backward reasoning



- Forward reasoning (or forward chaining): from initial states to goal states.

Er zijn twee manieren om een spelboom te doorzoeken, ofwel start je vanuit de initiële toestand en zoek je een weg naar een doeltoestand.
Ofwel probeer je vanuit een doeltoestand, in de omgekeerde richting, een pad te vinden naar de starttoestand. (volgende slide)
We focussen in deze cursus op forward reasoning.

Forward versus backward reasoning



- Backward reasoning (or backward chaining): from goal states to initial states.